ROHIT.L
192511286
CSA 0421
Operating System

CO1 - ASSESSMENT - TEST - 1

SHORT ANSWER

(1) Multiple Process in a shared Linux Server

a) A multi - user Operating system suitable for this environment

- It allows multiple user to access the same computer simultaneously.
- Each student can execute programs independently on the shared Linux server.
- It manages CPU time, memory, files, and other resources among different user.

b)

1. Process Management: The Kernel creates schedules and terminates process and allocates CPU time to them.

2. Memory Management: The Kernel allocates and deallocates memory for process.

(c)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    pid_t pid;
    pid = fork();
    if (pid < 0) {
        printf("Fork failed")
    } else if (pid == 0){
        printf("Child process %d\n", getpid());
        printf("Parent process %d\n", getppid());
    } else {
        printf("Parent Process %d\n", getpid());
        printf("Child Process %d\n", pid);
    }
    return 0;
}
```

(a)

1. creat() - Creates a file
2. open() - Opens a file
3. write() - Writes data
4. read() - Reads data
5. close() - Closes the file

(b)

1. open():
   - Open an existing file or access a file
   - Returns -1 if an error occurs

```
int fd = open("student.txt", O_RDONLY);
```

2. Write
   - writes data to a file
   - Returns the no. of file byte
   - Return -1 if an error occurs.

(c)

To create a file, student name, close the file.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    char name[] = "Raghel";
    fd = open("student.txt", O_WRONLY|O_TRUNC, 0644);
    int (fd == -1)
```

```
    perror ("File creation");
    return 1;
}
write(fd, name, strlen(name));
```

(3)

(a)
1. open()
2. read()
3. write()
4. close()

(b)
1. read()

Reads data from an opened file into memory buffer.

read (fd, buffer, sizeof(buffer));

2. close()

eg: close(fd);

(c)
```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
int main()
{
    int src, dest;
    char buf[1024];
    ssize_t n;
    src = open("source.txt", O_RDONLY);
    dest = open("dest.txt", O_WRONLY|0644);
    if (src == -1 || dest == -1)
```

- Collects the child's terminat
sta

```
        perror("File error");
        return 1;
    }
    while ((n = read(src, buf, sizeof(buf))) > 0)
        write(dest, buf, n);
```

4)

(a)

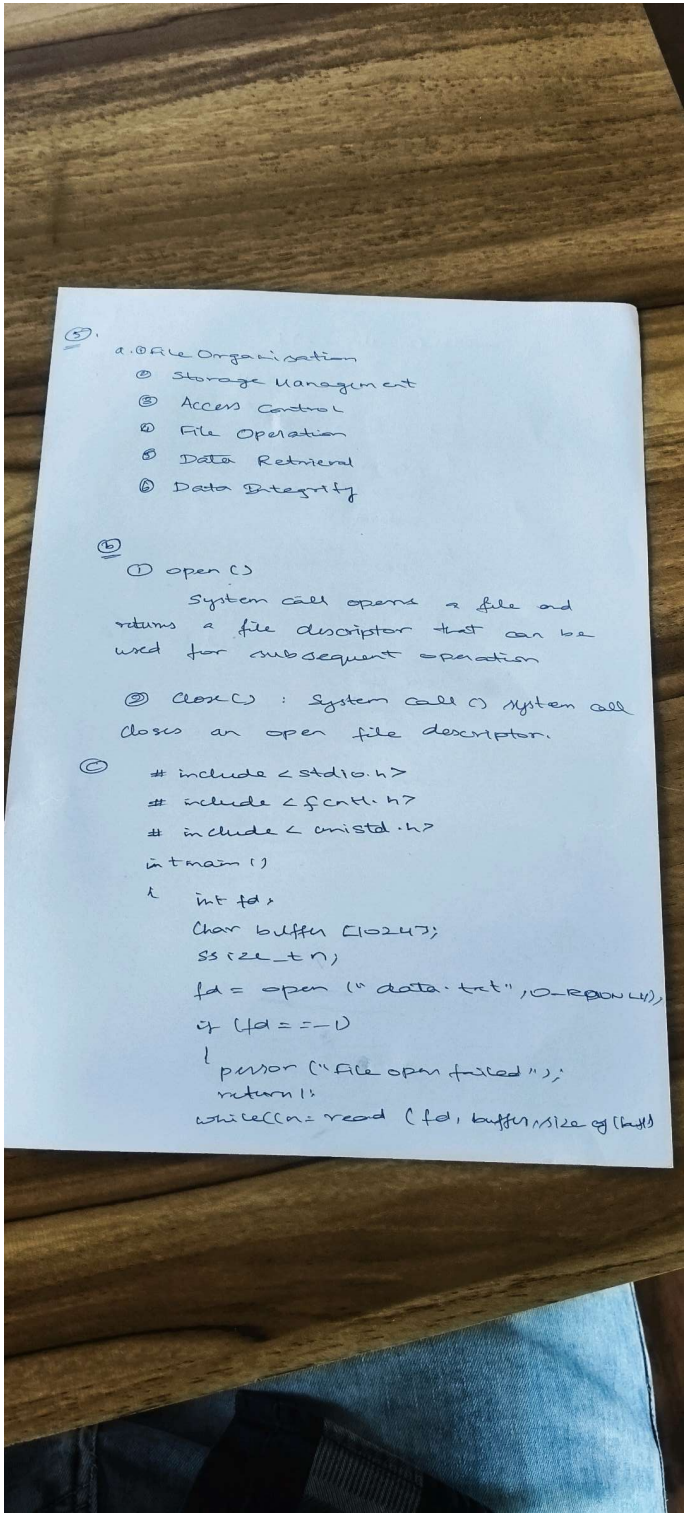
1. fork()
   - Creates a new child process from calling parent
   - Returns 0 to child process
   - Returns -1 to process if fails
2. execl()
   - Replaces the current process program with a new program
   - The process ID remains the same

(b) The wait() system call makes the parent process wait until one of its child process terminates.

- Prevent the parent from proceeding before child finishes.
- Collects the child's terminates status

(c)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0)
    {
    perror("Fork failed");
    return 1; }
    else if (pid == 0) {
        printf("Child process is executing")
        exit(1); } else
                wait(NULL);
                printf("Child process
                        completed");
            }
        return 0;
    }.
```



5.
a. ① File Organization
② Storage Management
③ Access Control
④ File Operation
⑤ Data Retrieval
⑥ Data Integrity

b.
① open()
System call opens a file and returns a file descriptor that can be used for subsequent operation

② close() : System call () system call closes an open file descriptor.

c.
```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    char buffer[1024];
    ssize_t n;
    fd = open("data.txt", O_RDONLY);
    if (fd == -1)
    {
        perror("file open failed");
        return 1;
    while((n = read(fd, buffer, sizeof(buffer)
```

```
while (written < n) {
    ssize_t w = write (STDOUT_FILENO,
                       buffer + written,
                       n - written);

    if (w <= 0)
    {
        perror ("Output failed");
        fclose (fd);
        return 1;
    }
    written += w;
}
if (n == -1)
    perror ("file read failed");

close (fd);
if (n == -1)
    return 1;

return 0;
}
```